

← این برگه را همراه دفترچه پاسخ تحویل دهید. نام و نام خانوادگی:

پرسش ۱ (۲ نمره) – تعریف کلاس‌های زیر را در نظر بگیرید و نتیجه کامپایل/اجرای هر یک از قطعه کدهای الف تا د را بنویسید.

<pre>class Base { public: virtual void f() { cout << 1; } virtual void g() { cout << 2; } void h() { g(); } }; class Derived : public Base { public: virtual void f(int x) { cout << x; } void g() { cout << 3; } };</pre>	(الف) <pre>Base *b = new Derived(); b->f(4); Compile Error</pre>
	(ب) <pre>Base *b = new Derived(); b->g(); 3</pre>
	(ج) <pre>Derived *d = new Derived(); d->h(); 3</pre>
	(د) <pre>Derived *d = new Base(); d->h(); Compile Error</pre>

پرسش ۲ (۳ نمره) – (الف) خروجی برنامه زیر را مشخص کنید.

```
#include <memory>
#include <iostream>
using namespace std;

class X {
public:
    X() { cout << "constr" << endl; }
    ~X() { cout << "destr" << endl; }
};
```

```
int main() {
    cout << "hello p" << endl;
    unique_ptr<X> p = make_unique<X>();
    {
        cout << "hello q" << endl;
        unique_ptr<X> q = move(p);
        cout << "bye q" << endl;
    }
    cout << "bye p" << endl;
}
```

hello p
constr
hello q
bye q
destr
bye p

(ب) استفاده از shared_ptr محدودیت‌های کمتری نسبت به unique_ptr دارد. پس چرا unique_ptr در زبان سی‌پلاس‌پلاس وجود دارد؟

به دلیل این که سربار زمانی ناشی از شمارش ارجاع‌ها (reference counting) را ندارد.

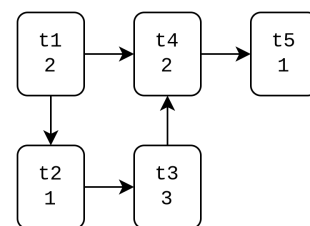
پرسش ۳ (۳ نمره) – فرض کنید تابع `void g()` ممکن است در جریان اجرای خود یکی از دو تاپی استثناء `FatalEx` یا `TimeoutEx` را پرتاب کند. تابعی به نام `f` بنویسید که `g` را فراخوانی می‌کند. در صورتی که استثناء `TimeoutEx` رخ دهد حداکثر تا ۱۰ بار تلاش می‌کند تابع `g` را اجرا کند. اگر هر ۱۰ بار با استثناء `TimeoutEx` مواجه شود، آخرین استثناء را برای تابع قبل پرتاب می‌کند. در صورتی که در جریان فراخوانی‌ها استثناء `FatalEx` رخ دهد، همان استثناء به تابع قبل پرتاب می‌شود.

```
void f(int n) {
    for (int i = 0; i < n; i++) {
        try {
            g();
            cout << "success in " << i << endl;
            return;
        } catch (FatalEx fe) {
            throw;
        } catch (TimeoutEx te) {
            if (i == n-1) {
                cout << "failure" << endl;
                throw;
            }
        }
    }
}
```

پرسش ۴ (۴ نمره) – فرض کنید قرار است در یک پروژه تعدادی وظیفه انجام شود. انجام هر وظیفه مدت زمان مشخصی طول می‌کشد (عدد صحیح بزرگتر از صفر). به علاوه، هر وظیفه تعدادی وظیفه دیگر را به عنوان پیش‌نیاز خود دارد. کلاسی به نام `Task` تعریف کنید که در سازنده خود نام وظیفه و مدت انجام آن را می‌گیرد. با متد `addPre` می‌توان وظیفه‌ای را به عنوان پیش‌نیاز وظیفه دیگر اضافه کرد. فرض کنید در گراف پیش‌نیازهای دور نخواهیم داشت. متد `earliest` زودترین زمانی را برمی‌گرداند که یک وظیفه می‌تواند به اتمام برسد (با فرض این که بتوانیم از زمان صفر اجرای پروژه را شروع کنیم). قطعه کد زیر مثالی از استفاده از این کلاس را برای پروژه مشخص‌شده در شکل نشان می‌دهد.

<code>Task t1("t1", 2);</code>	<code>t2.addPre(&t1);</code>
<code>Task t2("t2", 1);</code>	<code>t3.addPre(&t2);</code>
<code>Task t3("t3", 3);</code>	<code>t4.addPre(&t3);</code>
<code>Task t4("t4", 2);</code>	<code>t4.addPre(&t1);</code>
<code>Task t5("t5", 1);</code>	<code>t5.addPre(&t4);</code>

`t5.earliest() == 9`



کلاس `Task` را به طور کامل بنویسید.

```

class Task {
public:
    Task(string t, int d) : title(t), duration(d) {}
    void addPre(Task* p) { pre.push_back(p); }
    int earliest() {
        int max_earliest = 0;
        for (Task* tp : pre) {
            int pe = tp->earliest();
            if (pe > max_earliest)
                max_earliest = pe;
        }
        return max_earliest + duration;
    }
private:
    string title;
    int duration;
    vector<Task*> pre;
};

```

پرسش ۵ (۳ نمره) – مشکلات (زمان کامپایل یا اجرای) کلاس MyClass را که به شکل زیر تعریف شده مشخص کنید.

```

class MyClass {
public:
    MyClass(int s) : size(s) { p = new int[size]; }
    ~MyClass() { delete p; } // delete[] p;
    MyClass(const MyClass& other) : p(new int[(*other).size]) {} // محتوای آرایه باید کپی شود
                                                                // *other -> other

    MyClass& operator=(const MyClass& other) { /* correctly implemented */ }
private:
    int size;
    int* p;
};

```

پرسش ۶ (۵ نمره) – (الف) تابعی به نام count_if بنویسید که تعداد عناصری از یک container را برگرداند که یک شرط (predicate) را ارضاء می‌کنند. به عنوان مثال نتیجه اجرای کد زیر ۲ خواهد بود.

```

bool is_odd(int x) { return x%2; }
vector<int> v = {1, 2, 3};
cout << count_if(v.begin(), v.end(), is_odd);

```

این تابع را طوری تعریف کنید که برای تمام containerها و برای تمام تایپ‌های عناصر قابل استفاده باشد. پارامتر مربوط به شرط را طوری تعریف کنید که در استفاده از آن بتوان از اشاره‌گر به تابع، عبارت لامبدا یا شیء تابع استفاده کرد.

```

template<typename I, typename P>
int count_if(I first, I last, P pred) {
    int count = 0;
    while (first != last) {
        if (pred(*first))
            count++;
        ++first;
    }
    return count;
}

```

(ب) به عنوان کاربردی از تابع `count_if`، تابعی با امضای زیر بنویسید که برداری از تاریخ‌های تولد را می‌گیرد و تعداد تاریخ‌هایی که در فصل `season` هستند را برمی‌گرداند. مقادیر `season` از ۱ تا ۴ است. کلاس `Date` مطابق آن چه در مثال‌های کلاس دیده‌اید تعریف شده و دارای یک متد `get_month()` است.

```
int count_in_season(const vector<Date>& bdays, int season);
```

در تعریف این تابع از عبارت لامبدا استفاده کنید.

```
int count_in_season(const vector<Date>& bdays, int season) {  
    return count_if(bdays.begin(), bdays.end(),  
                    [=](Date d){ return (d.get_month()-1) / 3 == (season-1); });  
}
```